

MARKET EXPRESS

Creación de centros de almacenamiento y distribución de la empresa con alcance mundial Market Express

Market Express es una plataforma ficticia de comercio electrónico y logística que planea su expansión a nuevos mercados a nivel mundial. Buscan realizar sus entregas en lapsos de entre 24 a 48hs.

Problema: para cumplir con sus tiempos de entrega, la empresa debe decidir cómo construir su infraestructura en cada país nuevo. Puede optar por un único centro de almacenamiento y distribución, idealmente en la capital si es que la población se encuentra centralizada en la zona, o bien construir múltiples centros más pequeños si la población está repartida o el país es muy grande y estas dadas las condiciones estructurales.

Entonces se van a analizar las siguientes variables a nivel mundial :

- *Población en cada país.

- *Superficies de distancias a recorrer.

- *Densidad poblacional, para conocer si hay población suficiente en el país.

- *Cantidad de idiomas facilita o no la instalación de 1 o mas centros.

- *Esperanza de vida (que significa buena calidad de vida, servicios, e infraestructura)

- *GNP y GNP per cápita para conocer los ingresos del país, y los ingresos per cápita.

Luego con este análisis se procederá a clasificar a los países según si se recomienda la instalación de "Centros Únicos", "Varios Centros", "No se recomienda instalación" teniendo en cuenta estas variables: 'Densidad_poblacional', 'LifeExpectancy', 'IsOfficial', 'Crecimiento_económico' y 'GNP_per_cápita'.

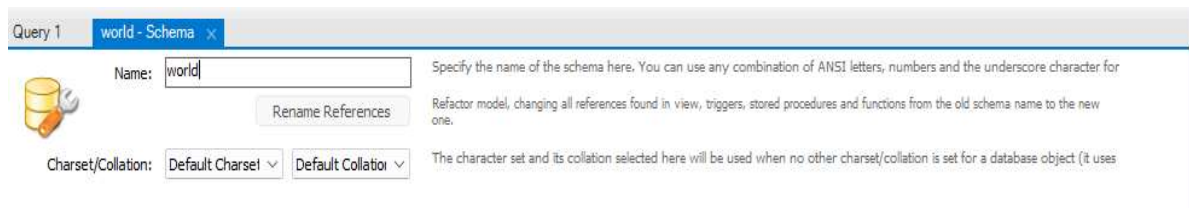
Por último se busca obtener un top 10 de países que se encuentren en estas mejores condiciones en conjunto, además de la recomendación de que tipo de centro instalar.

Para este proyecto se utiliza la base de datos World (Datos Mundiales) y las herramientas de sql, python y power bi para visualizaciones.

SECCIÓN 1

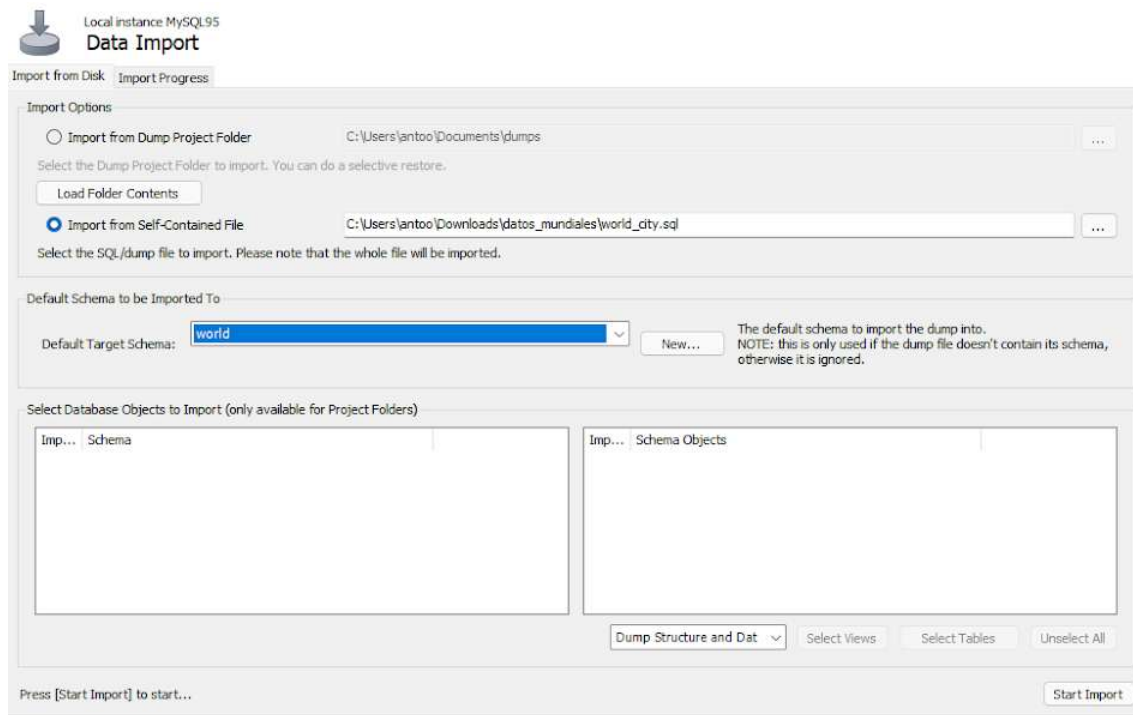
SQL: EXTRACCIÓN DE DATOS Y EXPLORACIÓN DE TABLAS.

Creación de base de datos llamada "world".



The screenshot shows the 'world - Schema' window in MySQL. It includes a 'Name' field with 'world' entered, a 'Charset/Collation' dropdown set to 'Default Charset' and 'Default Collation', and a 'Rename References' button. Explanatory text on the right states: 'Specify the name of the schema here. You can use any combination of ANSI letters, numbers and the underscore character for one.' and 'The character set and its collation selected here will be used when no other charset/collation is set for a database object (it uses the default for the server)'.

Importación de tablas para la base de datos: desde data import se importan las tablas city, country, y countrylanguage.



The screenshot shows the 'Data Import' wizard for a 'Local instance MySQL95'. The 'Import from Disk' tab is active. Under 'Import Options', 'Import from Self-Contained File' is selected with the file path 'C:\Users\antoo\Downloads\datos_mundiales\world_dty.sql'. The 'Default Target Schema' is set to 'world'. At the bottom, there are buttons for 'Dump Structure and Dat', 'Select Views', 'Select Tables', 'Unselect All', and a 'Start Import' button. A note at the bottom left says 'Press [Start Import] to start...'.



Local instance MySQL95

Data Import

Import from Disk

Import Progress

Import Options

☐ Import from Dump Project Folder

C:\Users\jantoo\Documents\dumps

...

Select the Dump Project Folder to import. You can do a selective restore.

Load Folder Contents

☒ Import from Self-Contained File

C:\Users\jantoo\Downloads\datos_mundiales\world_country.sql

...

Select the SQL/dump file to import. Please note that the whole file will be imported.

Default Schema to be Imported To

Default Target Schema:

world

▼

New...

The default schema to import the dump into.
NOTE: this is only used if the dump file doesn't contain its schema, otherwise it is ignored.

Select Database Objects to Import (only available for Project Folders)

Imp... Schema

Imp... Schema Objects

Dump Structure and Dat

Select Views

Select Tables

Unselect All

Import Completed

Start Import



Local instance MySQL95

Data Import

Import from Disk

Import Progress

Import Options

☐ Import from Dump Project Folder

C:\Users\jantoo\Documents\dumps

...

Select the Dump Project Folder to import. You can do a selective restore.

Load Folder Contents

☒ Import from Self-Contained File

C:\Users\jantoo\Downloads\datos_mundiales\world_countrylanguage.sql

...

Select the SQL/dump file to import. Please note that the whole file will be imported.

Default Schema to be Imported To

Default Target Schema:

world

▼

New...

The default schema to import the dump into.
NOTE: this is only used if the dump file doesn't contain its schema, otherwise it is ignored.

Select Database Objects to Import (only available for Project Folders)

Imp... Schema

Imp... Schema Objects

Dump Structure and Dat

Select Views

Select Tables

Unselect All

Import Completed

Start Import

EXPLORACIÓN DE TABLAS

Se revisan los primeros 10 registros de cada tabla, sus tipos de datos y duplicados.

CITY

```
1 • USE world;
2 • SELECT * FROM CITY LIMIT 10;
```

Resultado:

	ID	Name	CountryCode	District	Population
▶	1	Kabul	AFG	Kabul	1780000
	2	Qandahar	AFG	Qandahar	237500
	3	Herat	AFG	Herat	186800
	4	Mazar-e-Sharif	AFG	Balkh	127800
	5	Amsterdam	NLD	Noord-Holland	731200
	6	Rotterdam	NLD	Zuid-Holland	593321
	7	Haag	NLD	Zuid-Holland	440900
	8	Utrecht	NLD	Utrecht	234323
	9	Eindhoven	NLD	Noord-Brabant	201843
	10	Tilburg	NLD	Noord-Brabant	193238

Tipos de Datos de cada columna:

Field Types								
	Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale
1	ID	world	city	INT	binary	11	2	0
2	Name	world	city	CHAR	utf8mb4	35	14	0
3	CountryCode	world	city	CHAR	utf8mb4	3	3	0
4	District	world	city	CHAR	utf8mb4	20	13	0
5	Population	world	city	INT	binary	11	7	0

Duplicados de columna Name:

```
2 • SELECT
3     Name,
4     COUNT(*) AS cantidad_duplicados
5 FROM CITY
6 GROUP BY Name
7 HAVING COUNT(*)>1
```

Resultado, hay ciudades duplicadas:

Name	cantidad_duplicados
Ede	2
Saint John 's	2
Córdoba	3
San Miguel	3
San Fernando	3
San Juan	2
Newcastle	2
Hamilton	3
La Paz	3
Santa Maria	2

Para verificarlo, se ve el primer ejemplo de la ciudad "Ede", pero nos demuestra que pertenecen a distintos países, por lo que es correcto que figure en este caso 2 veces.

```
2 • SELECT *
3 FROM CITY
4 WHERE Name='Ede';
```

ID	Name	CountryCode	District	Population
29	Ede	NLD	Gelderland	101574
2759	Ede	NGA	Oyo & Osun	307100

Pero al buscar duplicados por ID no se encuentra ninguno:

```
2 • SELECT
3 ID,
4 COUNT(*) AS cantidad_duplicados
5 FROM CITY
6 GROUP BY ID
7 HAVING COUNT(*)>1
```

ID	cantidad_duplicados
----	---------------------

COUNTRY



```
1 • USE world;
2 • SELECT * FROM COUNTRY LIMIT 10;
```

Resultado:

Code	Name	Continent	Region	SurfaceArea	IndepYear	Population	LifeExpectancy	GNP	GNPOld	LocalName
ABW	Aruba	North America	Caribbean	193.00	NULL	103000	78.4	828.00	793.00	Aruba
AFG	Afghanistan	Asia	Southern and Central Asia	652090.00	1919	22720000	45.9	5976.00	NULL	Afganistan/A
AGO	Angola	Africa	Central Africa	1246700.00	1975	12878000	38.3	6648.00	7984.00	Angola
AIA	Anguilla	North America	Caribbean	96.00	NULL	8000	76.1	63.20	NULL	Anguilla
ALB	Albania	Europe	Southern Europe	28748.00	1912	3401200	71.6	3205.00	2500.00	Shqipëria
AND	Andorra	Europe	Southern Europe	468.00	1278	78000	83.5	1630.00	NULL	Andorra
ANT	Netherlands Antilles	North America	Caribbean	800.00	NULL	217000	74.7	1941.00	NULL	Nederlandse
ARE	United Arab Emirates	Asia	Middle East	83600.00	1971	2441000	74.1	37966.00	36846.00	Al-Imarat al-
ARG	Argentina	South America	South America	2780400.00	1816	37032000	75.1	340238.00	323310.00	Argentina
ARM	Armenia	Asia	Middle East	29800.00	1991	3520000	66.4	1813.00	1627.00	Hajastan

Se comprueba si hay países duplicados en la columna Code y luego en Name pero no se encuentra ninguno:

```
4 • SELECT
5 Code,
6 COUNT(*) AS cantidad_duplicados
7 FROM COUNTRY
8 GROUP BY Code
9 HAVING COUNT(*)>1;
```

Code	cantidad_duplicados
------	---------------------

```

11 • SELECT
12     Name,
13     COUNT(*) AS cantidad_duplicados
14 FROM COUNTRY
15 GROUP BY Name
16 HAVING COUNT(*)>1;

```

Name	cantidad_duplicados
------	---------------------

Tipo de datos:

Field Types								
Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale	
1 Code	world	country	CHAR	utf8mb4	3	3	0	
2 Name	world	country	CHAR	utf8mb4	52	20	0	
3 Continent	world	country	ENUM	utf8mb4	13	13	0	
4 Region	world	country	CHAR	utf8mb4	26	25	0	
5 SurfaceArea	world	country	DECIMAL	binary	12	8	2	
6 IndepYear	world	country	SMALLINT	binary	6	4	0	
7 Population	world	country	INT	binary	11	8	0	
8 LifeExpectancy	world	country	DECIMAL	binary	5	3	1	
9 GNP	world	country	DECIMAL	binary	12	7	2	
10 GNPOld	world	country	DECIMAL	binary	12	7	2	
11 LocalName	world	country	CHAR	utf8mb4	45	35	0	
12 GovernmentForm	world	country	CHAR	utf8mb4	45	44	0	
13 HeadOfState	world	country	CHAR	utf8mb4	60	27	0	
14 Capital	world	country	INT	binary	11	3	0	
15 Code2	world	country	CHAR	utf8mb4	2	2	0	

COUNTRYLANGUAGE

Don't Limit

```

1 • USE world;
2 • SELECT * FROM COUNTRYLANGUAGE LIMIT 10;

```

Resultado:

CountryCode	Language	IsOfficial	Percentage
ABW	Dutch	T	5.3
ABW	English	F	9.5
ABW	Papiamentu	F	76.7
ABW	Spanish	F	7.4
AFG	Balochi	F	0.9
AFG	Dari	T	32.1
AFG	Pashto	T	52.4
AFG	Turkmenian	F	1.9
AFG	Uzbek	F	8.8
AGO	Ambo	F	2.4
NULL	NULL	NULL	NULL

Se analiza si hay lenguajes duplicados:

```
4 • SELECT
5     Language,
6     COUNT(*) AS cantidad_duplicados
7 FROM COUNTRYLANGUAGE
8 GROUP BY Language
9 HAVING COUNT(*)>1;
```

Se puede ver que si hay lenguajes que pueden utilizarse en varios países. Se ve el ejemplo "Dutch" que se utiliza en 5 países:

```
11 • SELECT *
12 FROM COUNTRYLANGUAGE
13 WHERE Language = 'Dutch';
```

CountryCode	Language	IsOfficial	Percentage
ABW	Dutch	T	5.3
ANT	Dutch	T	0.0
BEL	Dutch	T	59.2
CAN	Dutch	F	0.5
NLD	Dutch	T	95.6
NULL	NULL	NULL	NULL

Tipos de datos:

Field Types								
Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale	
1 CountryCode	world	country/language	CHAR	utf8mb4	3	3		0
2 Language	world	country/language	CHAR	utf8mb4	30	10		0
3 IsOfficial	world	country/language	ENUM	utf8mb4	1	1		0
4 Percentage	world	country/language	DECIMAL	binary	6	3		1

SECCIÓN 2

PYTHON: EXTRACCIÓN, LIMPIEZA Y TRANSFORMACIÓN DE DATOS.

Instalación de librería sqlalchemy, conexión con sql y creación de Dataframes

- Primero se instalan las librerías python-dotenv, sqlalchemy y cryptography, para poder lograr la conexión con SQL y dejar las credenciales ocultas en un archivo .env.
- Luego se importan os, load_dotenv, create_engine, pandas y numpy.
- Una vez instaladas, se crean los dataframes: df_city, df_country y df_cl, para cada uno de ellos se utiliza un pd.read_sql para traer toda la información de las tablas country, city y countrylanguage.

Exploración de dataframes

- Se analiza cada dataframe con .info() y .describe() para revisar nulos y datos estadísticos como count, mean, std, min, max y cuartiles. Se encuentra dentro de df_country que LifeExpectancy y GNPOld (columnas relevantes para el análisis) tienen nulos.

Tratamiento de nulos

- Se crea una copia de df_country llamada df_country2 para no modificar el dataframe original.
- Se rellena a los nulos de LifeExpectancy (columna relevante para el análisis) con la media de LifeExpectancy utilizando .fillna() y np.mean().
- Se repite lo mismo con GNPOld, se rellenan los nulos con su valor correspondiente de GNP. Luego se confirma con info que ya no están los nulos en estas columnas de df_country2 y se vuelve a ver las estadísticas con .describe().

Creación de columnas calculadas necesarias para el análisis

Se crean las siguientes columnas en df_country2:

- 'Crecimiento_económico': si $GNP > GNPOld$, se devuelve 'Si' y de lo contrario 'No'. Se utiliza función np.where().
- 'GNP_per_cápita': se multiplica el GNP de cada país por 1000000 y se lo divide por Population de cada país.
- 'Densidad_poblacional': se divide Population de cada país con SurfaceArea de cada uno.

Análisis población

- Primero se actualiza Population de China a 1500000000, según la Consigna 1, utilizando la función .loc().
- Se ordena de forma descendente df_country2 por Population, trayendo también Name. Utilizando .sort_values().
- Se eligen los 10 países con mayor Populations utilizando función .nlargest().
- Se consulta Population de los países de Europa, también solicitado en Consigna 1.
- Mediante la función lambda, se cambia en Continent , North América y South América por América para poder unificarlas en un continente.
- Se consulta la columna Population y Name filtrando por cada continente por separado.

- Se agrupa el df_country2 por Continent (usando .groupby()), y se suma Population de cada continente (función .agg()). Solicitado en la consigna 3. También se realizó el mismo procedimiento agregando países.
- La Consigna 4 solicita traer las ciudades y su población por país. Se crea el df_join para poder unir mediante pd.merge los df_city y df_country. Luego se renombran las columnas mediante .rename() 'Name_x' a 'City', 'Population_x' a 'Population_city', 'Name_y' a 'Country', y 'Population_y' a 'Population_country'.
- Se crea un dataframe llamado Ciudades_europa utilizando el df_join, filtrando como continente Europa y trayendo City y Population_city, ordenadas por Population_city de forma descendente.

Análisis Superficie

- Según la Consigna 2 se obtienen los 5 países con mayor SurfaceArea, a df_country2 se le aplica .nlargest() para conseguirlo.
- Se consulta la SurfaceArea de cada continente, agrupando por cada uno de ellos. Y realizando una suma de SurfaceArea, se los ordena de forma descendente.
- Se calcula la media de SurfaceArea utilizando .mean().

Análisis Densidad Poblacional

- Desde df_country2 se calcula la Densidad_poblacional media (.mean()).
- También se calcula la Densidad_poblacional total por continente y se ordena de forma descendente utilizando .groupby(), .sum(), .sort_values().
- Se consulta la Densidad_poblacional por país, como se solicita en la Consigna 8.

Análisis de Idiomas

- Para averiguar el idioma oficial de cada país se crea df_lan_oficial y se filtra desde df_cl aquellos países trayendo Countrycode y Language, donde IsOfficial es T.
- Luego se crea df_cl2 par unir con pd.merge a df_join y df_cl para poder obtener los países y el recuento de idiomas que tiene cada uno de forma descendente. Utilizando .groupby(), .nunique() y .sort_values(). Luego se crea Cont_lan desde un .groupby() por continente para poder obtener lenguajes únicos, como es solicitado en la Consigna 10.
- En la Consigna 6 se solicita conocer cuáles y cuantos son los idiomas oficiales de South America. Se filtra a df_cl2 por IsOfficial cuando es T, y Continent cuando es South America. Trayendo solo Language con .unique() y .nunique().
- Se realiza el mismo procedimiento para consultar por IsOfficial cuando es F para South America. También es que se descubre que hay países que no tienen idiomas oficiales.
- Luego en df_max_lan2 se calculan los países donde los porcentajes de idioma son más altos utilizando .idxmax() y .sort_values() de forma descendente.

Análisis Condiciones de Vida

- En la Consigna 7 se solicita mostrar a los países con LifeExpectancy>75, los datos se traen desde el df_country2 y se filtra por lo solicitado trayendo también la columna Name. Luego se cuentan con .count() estos países, y se calcula su media con .mean().
- Se calcula el país con mayor LifeExpectancy siendo max_esp2 utilizando .idxmax().
- Se obtiene LifeExpectancy media por continente, utilizando .groupby() y .mean().

Análisis de GNP y GNP Per cápita

- Primero se consulta qué países tuvieron Crecimiento_economico, filtrando por Si en df_country2, y luego se hace un recuento con .count().
- También se consulta que países tuvieron Crecimiento_economico en Europa y América, filtrando por ambos a df_country2 y haciendo un recuento con .count().
- Luego se calcula el GNP_per_cápita medio a nivel mundial con .mean() y luego por continente ordenados de forma descendente.

Unión de Dataframes

- Se crea df_country3 para realizar un merge (left join) con df_country2 y df_city mediante Capital e ID. Luego se crea df_IsOfficial desde df_cl para poder obtener todos aquellos lenguajes con la condición T.
- Además se une en df_final, mediante un merge (left join) df_country3 y df_IsOfficial.
- Se borra de df_final con .drop() las columnas de CountryCode_x, CountryCode_y Capital ya que duplicaban la información.
- Se renombran con .rename() las columnas de 'Name_x' a 'Country', 'Population_x' a 'Population_country', 'Name_y' a 'Capital', 'Population_y' a 'Population_capital'.
- Luego se busca borrar los duplicados de países y dejar solo aquellos que tengan su ciudad capital y mayor porcentaje de lenguaje IsOfficial igual a T. Para esto en df_final2 se convierte Percentage con pd.to_numeric() para asegurarse que esté en formato número y se los ordena por CountryCode y Percentage de forma descendente. Por último se borran los duplicados con .drop_duplicates().
- Se crea la variable fill_na_lan para completar los nulos que figuren en las siguientes columnas de df_final2: 'Language' a 'No Official', 'Is Official' a 'F', 'Percentage' a 'Na', utilizando .fillna().

Clasificación de países y sus tipos de centros

- Primero se crea cols_validar con los valores de Densidad_Poblacional, LifeExpectancy, GNP, GNPOld, y GNP_per_cápita y mediante una función apply y pd.numeric se asegura de que sean valores numéricos.
- Se crean las variables para la clasificación: condicion_centro_unico donde se realiza un filtro de países teniendo en cuenta 'Densidad_poblacional' > 409.86, 'LifeExpectancy' > 66, 'IsOfficial' == 'T', 'Crecimiento_económico' == 'Si', y 'GNP_per_cápita' > 6089.69. En este caso, se toman para el análisis aquellos países que superen estos valores que resultan de la media de cada factor ya calculados anteriormente.
- También está la variable para clasificación de: condicion_multiples teniendo en cuenta 'Densidad_poblacional' > 50, 'LifeExpectancy' > 53, 'IsOfficial' == 'T', 'Crecimiento_económico' == 'Si', y 'GNP_per_cápita' > 500. En este caso, se toman para el análisis aquellos países que superen estos valores.
- Se crea una clasificación para condicion_centro_unico, y condicion_multiples para obtener como resultados: 'Centro Único' y Múltiples Centros'.
- Se agrega una columna a df_final2 llamada 'Recomendacion_Instalacion', para aplicar la clasificación anterior a cada país, y en el caso que no se cumplan que devuelva 'No se recomienda instalación'. Se utiliza para esto np.select().
- Se revisa cuantos países corresponden a cada clasificación mediante .value_counts().

- Se convierten las columnas de SurfaceArea, IndepYear, ID, Population_country y Population_capital en enteros con .astype(), y las columnas de LifeExpectancy, GNP, GNP_per_capita y Densidad_poblacional en decimales.
- Se ordena df_final2 por GNP_per_cápita, LifeExpectancy y Densidad_poblacional de forma descendente y se lo descarga .csv con la clasificación ya realizada.

Filtro de top 10 países recomendados para la instalación de centros

- Se toma los primeros diez países de df_final2 .head() con sus respectivos valores.

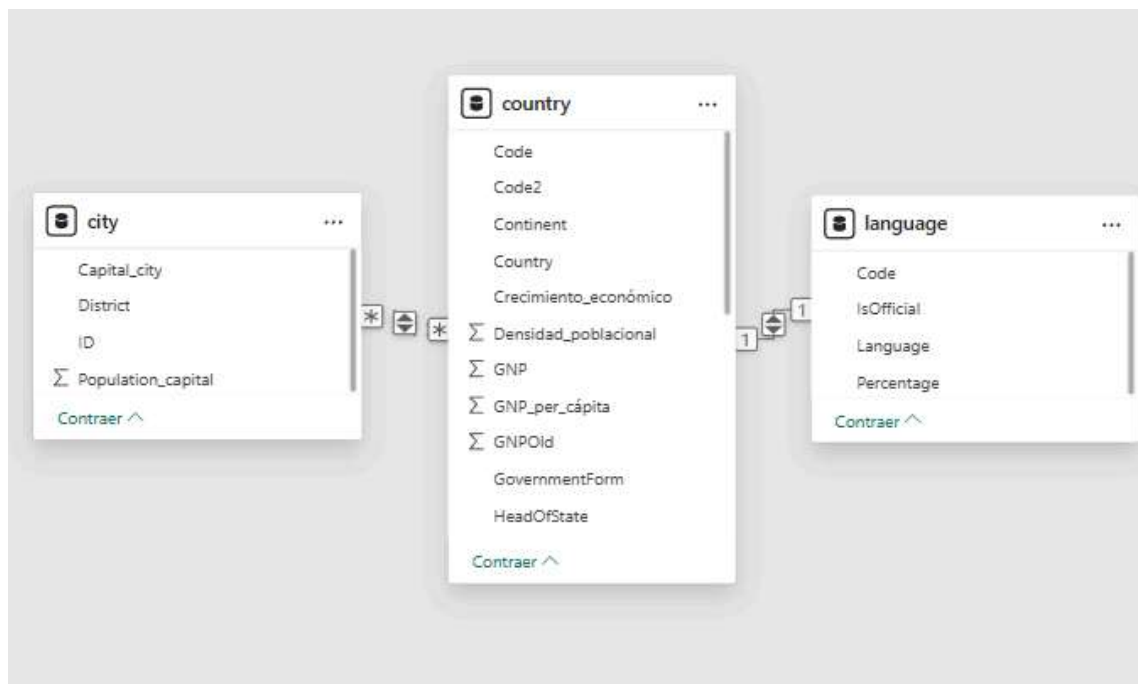
Descarga de documentos

- Además de df_final2 con la clasificación del conjunto de países, se descargan en formato csv df_country2, df_city y df_cl.

SECCIÓN 4

POWER BI

- Se utiliza power bi desktop para mostrar las visualizaciones del proyecto. Para esto se importó df_final2 y a partir de allí se lo volvió a separar en las tablas: city, country y language, se crearon también las relaciones desde del modelo.



- En la solapa Continentes se puede encontrar un segmentador de los mismos, pudiendo seleccionar cada uno de ellos y logrando así que en determinadas tarjetas se muestre la cantidad total y media de la población expresada en millones, la densidad poblacional media, el GNP per cápita total y medio, la cantidad de países y su esperanza de vida media. También se encuentran dos gráficos estáticos que no dependen del segmentador: un gráfico circular con la cantidad de lenguajes por continente y uno de dispersión entre población y superficie.

- En la solapa Países pueden encontrarse: un gráfico de columnas donde puede verse un top 10 de países con mayor GNP per cápita, dos gráficos de barras que muestran las ciudades con más de 5 millones de habitantes y el top 10 países con mayor densidad poblacional, una tarjeta con las medias de densidad poblacional, esperanza de vida y GNP per cápita. Por último se ve un tree map del top 10 países de mayor GNP per cápita y su esperanza de vida media.
- En la solapa Top 10 países, podemos encontrar un segmentador con el top 10 países con mayor GNP per cápita. A partir de allí nos muestra los indicadores clave de los mismos. En tarjetas se encuentra: su superficie en km2, GNP total, población, continente, idioma oficial y el tipo de centro recomendado. En medidores se puede ver la densidad poblacional, esperanza de vida y gnp per cápita en relación a la media. Además hay un mapa del país.
- Para los visualizaciones se crearon las siguientes medidas DAX:
 - Continente seleccionado = `SELECTEDVALUE(country[Continent])`
 - País seleccionado = `SELECTEDVALUE(country[Country])`
 - Mínima Esperanza de Vida =
`CALCULATE(
 MINX(ALL('country'), [LifeExpectancy]))`
 - Máxima Esperanza de Vida =
`CALCULATE(
 MAXX(ALL('country'), [LifeExpectancy]))`
 - Mínimo GNP per cápita =
`CALCULATE(
 MINX(ALL('country'), [GNP_per_cápita]))`
 - Máximo GNP per cápita =
`CALCULATE(
 MAXX(ALL('country'), [GNP_per_cápita]))`
 - Mínima Densidad Poblacional = `MIN(country[Densidad_poblacional])`
 - Media Densidad Poblacional =
`CALCULATE(
 AVERAGEX(ALL('country'), [Densidad_poblacional]))`